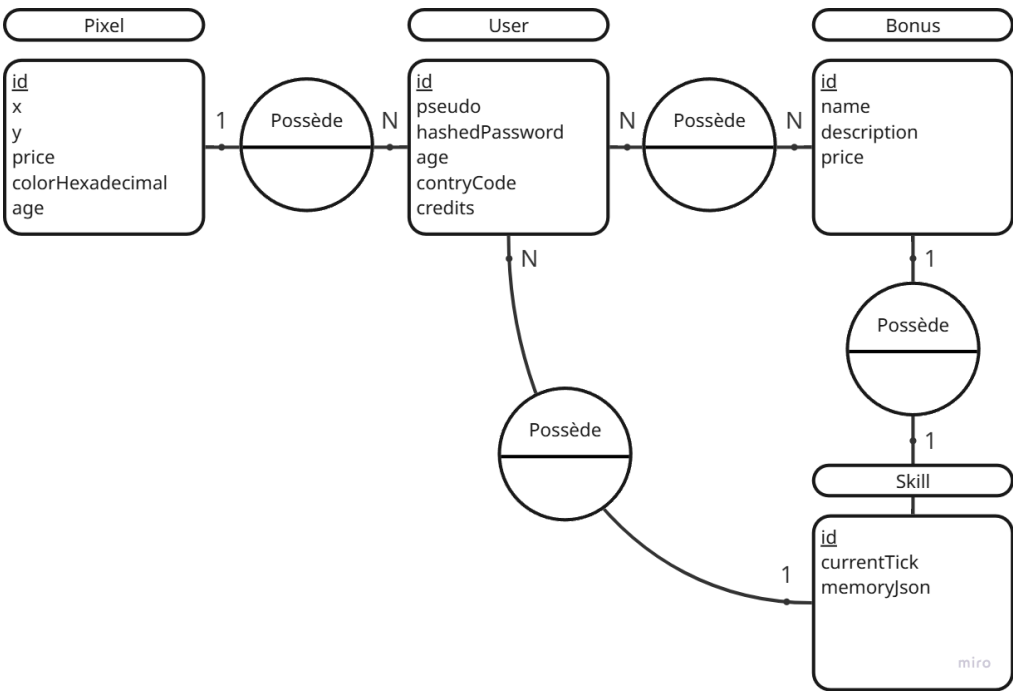


Base de données	2
Backend	3
Architecture	3
Les points d'entrées	5
Les services	6
Les accès externes	7
Le Jeu	8
Les événements	9
Websocket	10
Frontend	11

Base de données

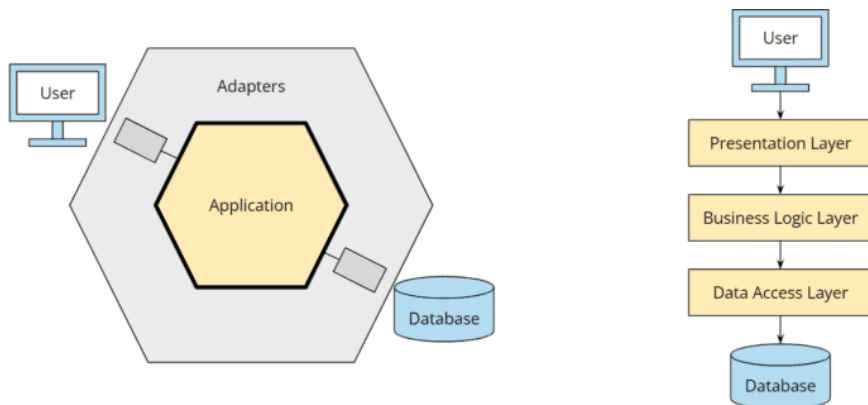


Backend

Architecture

Le backend suit une architecture en couche, inspirée par des concepts de l'architecture hexagonale. Les deux architectures se ressemblent mais utilisent un formalisme différent. (on va le voir sur l'image en dessous)

Pour y voir les différences, prenons comme exemple une application qui tournerait à 100% en local.



Parmi les formalismes qui changent :

- La couche présentation est un “adaptateur entrant”
- La couche d'accès aux données est un “adaptateur sortant”

Ce qui implique sémantiquement deux choses :

1. Les adaptateurs sont des ponts entre l'application et l'extérieur.
2. Les adaptateurs sont interchangeables (entre eux).

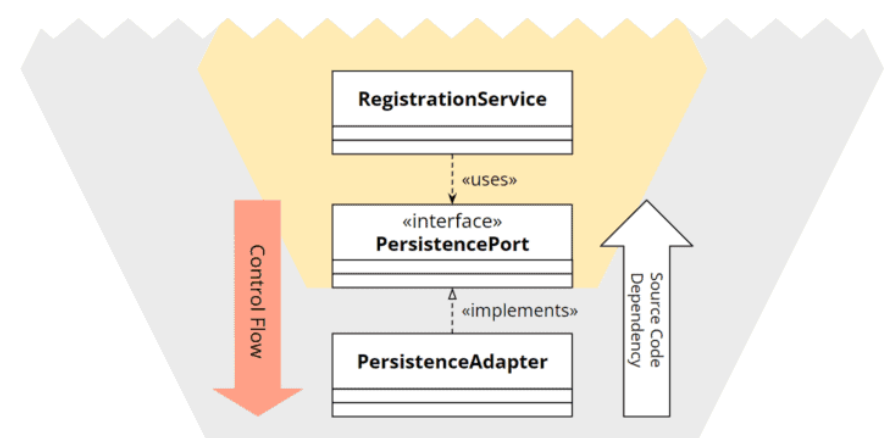
Pour interagir avec notre application on pourrait par exemple ajouter un adaptateur entrant qui gérerait le HTTP, pour exposer l'application à des routes. Ou bien un adaptateur entrant qui gèrerait un Websocket.

Pour persister nos données sur une nouvelle BD par exemple, on peut créer un nouvel adaptateur sortant que l'on branche à la place de l'ancien.

Pour mettre en place cette architecture nous avons respecté un sens de dépendance stricte :

- L'application dépend d'abstraction des adaptateurs sortant et les adaptateur entrant dépendent de l'application

Pour les adaptateurs sortant, nous avons donc utilisé l'inversion de dépendance (le D de SOLID).



Enfin pour les adaptateurs entrant, on appelle nos méthodes métiers directement.

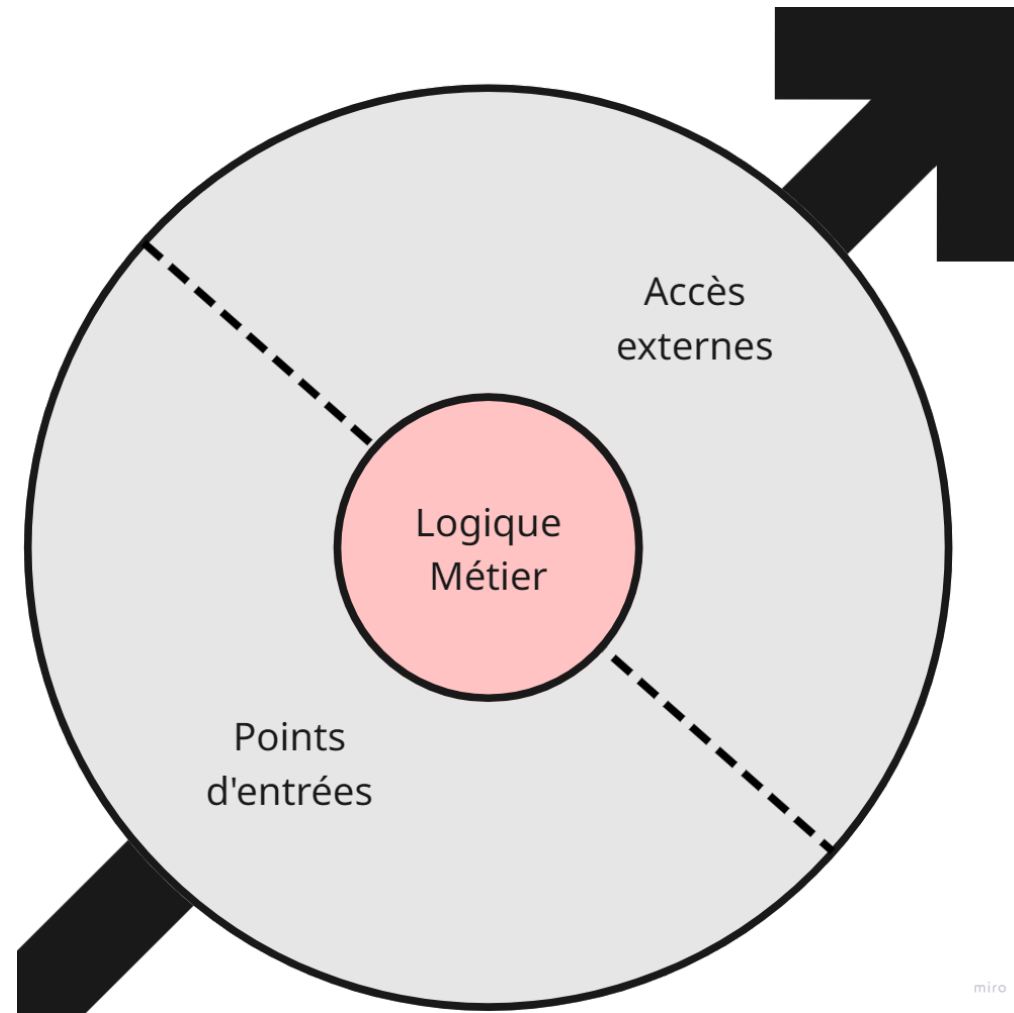
Toutes ces images proviennent de cet article :

<https://www.happycoders.eu/software-craftsmanship/hexagonal-architecture>

Cet article m'a inspiré :

<https://kaizen-solutions.net/kaizen-insights/articles-et-conseils-de-no-s-experts/introduction-a-l-architecture-hexagonale/>

Notre backend en simple :



Les points d'entrées

Dans le dossier /controllers se trouve toutes les classes servant de point d'entrée, via HTTP ou Websocket.

Leur seule responsabilité est de traduire une demande HTTP ou un événement Websocket (désérialisation / parsing) en appel métier, puis de répondre, au besoin, en utilisant le même "langage".

Exemple pour l'enregistrement d'un utilisateur :

/controllers/AccountHttpJavalin.java

```
46     public void postRegister(Context context) throws Exception {  
47         PostRegisterBody body = context.bodyAsClass(PostRegisterBody.class);  
48         if (!accountService.register(body.pseudo, body.password))  
49             throw new Exception("Pseudo déjà pris");  
50         context.status(200);  
51     }  
52 }
```

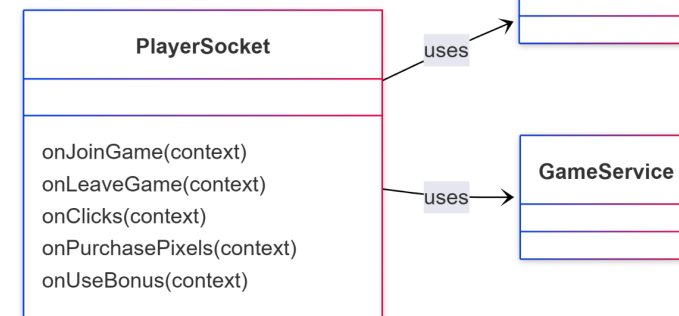
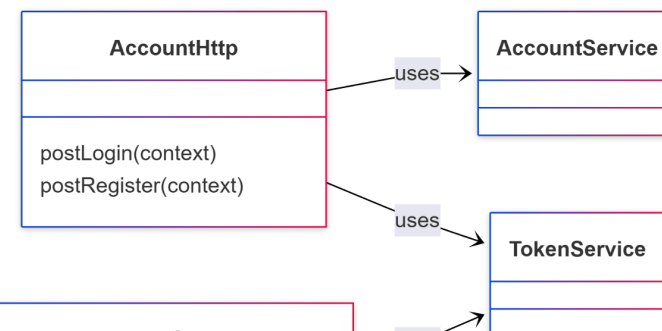
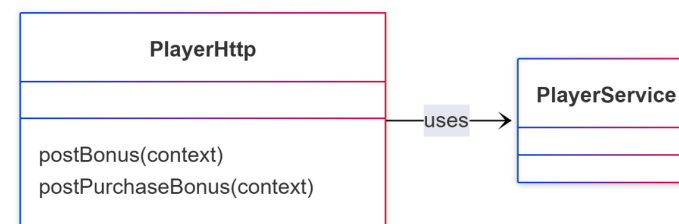
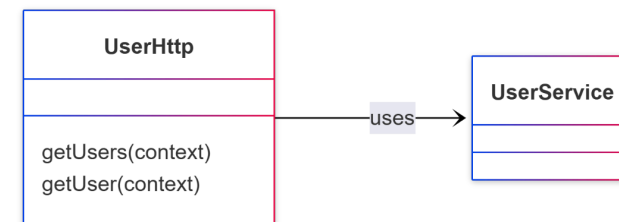


Diagramme UML - Contrôleurs

Les services

Les classes contenu dans /services contiennent la logique et les règles métiers de l'application.

Ces classes orchestrent les objets d'accès externe comme les DAO et les entités métiers interne comme **Game**.

Exemple pour l'utilisation d'un bonus :

/services/GameService.java

```
25     public void buyBonus(int userId, int bonusId) throws Exception {
26         User user = userDao.find(userId);
27         Bonus bonus = bonusDao.find(bonusId);
28
29         if (user == null)
30             throw new Exception("Utilisateur introuvable, userId invalide");
31         if (user.hasBonus(bonus))
32             throw new Exception("Le joueur possède déjà ce bonus");
33         if (user.getCredits() < bonus.getPrice())
34             throw new Exception("Le joueur n'a pas assez de crédits");
35
36         int price = bonus.getPrice();
37         user.removeCredits(price);
38         user.addBonus(bonus);
39         userDao.update(user);
40     }
41 }
```

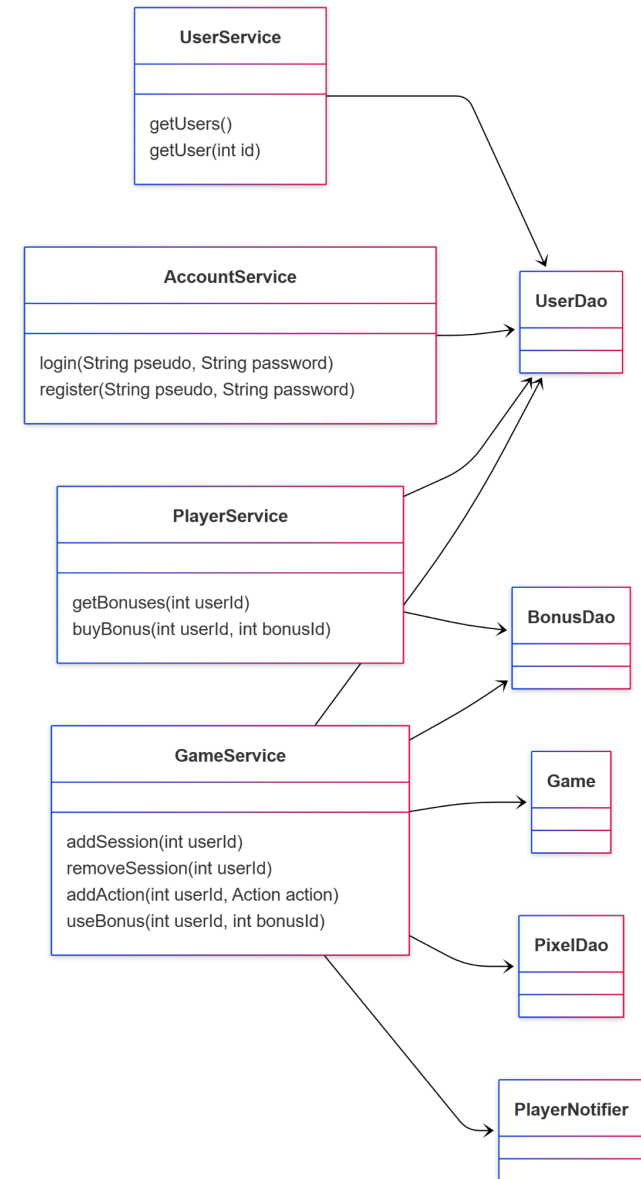


Diagramme UML - Services

Les accès externes

Toutes les classes ici implémentent des interfaces, comme expliqué plus haut, car ce sont des adaptateurs sortant.

Dans /notifiers on trouve l'interface **PlayerNotifier** responsable de transmettre une information vers le frontend, comme la mise à jour d'un pixel ou l'échec d'une action du joueur.

```
9  public interface PlayerNotifier {
10      public void playerUpdated(int userId, Player player);
11      public void pixelsUpdated(Set<Pixel> pixels);
12      public void skillStarted(int userId, SkillState skill);
13      public void skillEnded(int userId, SkillState skill);
14      public void actionFailed(int userId, String message);
15  }
```

Dans le dossier /daos se trouve les classes qui interagissent avec la persistance. Les classes DAO implémenter avec JPA sont responsables du dialogue entre la logique métier et les requêtes à la base de données.

Elles sont donc aussi responsables de transformer les objets métier en objets JPA (ORM) contenu dans /orms et inversement.

On trouve aussi dans ces dossiers des classes “mock” pour simuler une BD ou l'envoi de notification. Ces classes sont utiles pour les tests unitaires et pour le développement.

Pas besoin de lancer une base de données pour tester notre code par exemple.

```
9  public class BonusDaoMock implements BonusDao {
10      private Map<Integer, Bonus> storage = new HashMap<>();
11
12      public BonusDaoMock() {
13          storage.put(0, ...);
20         storage.put(1, ...);
27     }
28
29     @Override
30     public Bonus find(int id) {
31         return storage.get(id);
32     }
33
34     @Override
35     public Bonus[] every() {
36         return storage.values().toArray(new Bonus[0]);
37     }
38 }
```

Exemple d'un Mock

Le Jeu

Toute la logique en concurrence des joueurs est gérée dans la classe **Game** dans /game.

En effet, nous avons distingué les actions en concurrence, comme l'achat d'un pixel et les actions sans concurrence comme l'achat d'un bonus.

En fait, pour des raisons d'équité, tout ce qui mène à un gain de pixels dans le temps est traité comme une action en concurrence.

Les gains de crédits liés à un bonus passif, un bonus qui donnerait 10 crédits toutes les secondes par exemple. Car avec des crédits, on achète des pixels.

En tout cas **Game** est responsable de :

- Demander l'exécution des actions des joueurs dans leur ordre d'arrivée (FIFO).
- Demander l'application des skills sans favoritisme. (les skills sont des bonus qui possèdent des effets particuliers...)

L'exécution réel des actions est contenu dans les classes qui implémentent l'interface **Action**, dans le dossier /game/actions. Pareillement, l'application des bonus se fait dans les classes qui implémentent l'interface **Skill** dans le dossier /game/skills.

Nous devons aussi gérer des états pour notre jeu. La grille de pixel bien sûr, et même des états propres à un bonus en particulier.

Un bonus qui multiplierait le gain de crédit par deux pendant 20 secondes par exemple. Pour pouvoir le stopper au bon moment, nous devons savoir depuis combien de temps il est actif.

Les états du jeu sont contenus dans **GameState** et **SkillState** dans le dossier /game. Ces classes sont donc responsables de la modification de l'état du jeu.

Pour en revenir à l'application réel des actions et des bonus, la classe **Game** appelle des méthodes d'interfaces en leur fournissant en arguments : l'état du jeu, l'état du skill et l'identifiant du joueur concerné.

Note : Dans le code on fournit aussi un **GameEventSource**, on en reparle juste après...

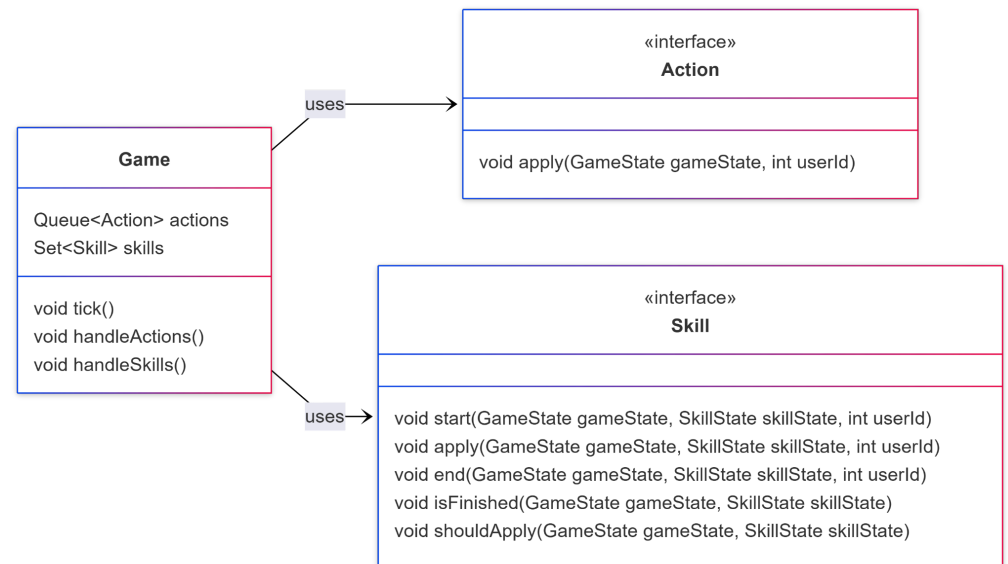


Diagramme UML - Game

Les événements

Quand un joueur achète un pixel, l'action qui correspond s'exécute et modifie l'état du jeu (via **GameState** et **SkillState**). Il va ensuite prévenir que l'état à changer, précisément, il prévient le joueur concerné qu'un pixel ses ajouté à sa liste de pixel et il prévient tous les joueurs actifs qu'un pixel à changé de propriétaire.

C'est pourquoi nous avons dû implémenter des événements. La classe **GameEventSource** déclenche les événements, **GameService** l'écoute et réagit en implémentant l'interface **GameEventListener**.

Cette classe et cette interface sont toutes deux contenues dans le dossier game/events.

Ensuite, comme vu précédemment dans [Les services](#), **GameService** transmet l'information aux classes externes. Dans l'exemple d'un achat de pixel, il demande au notifieur d'envoyer un message à tous les joueurs, en leur envoyant les informations du pixel qui vient de changer.

L'implémentation du patron est inspiré de cet article :

https://codemia.io/knowledge-hub/path/create_a_custom_event_in_java

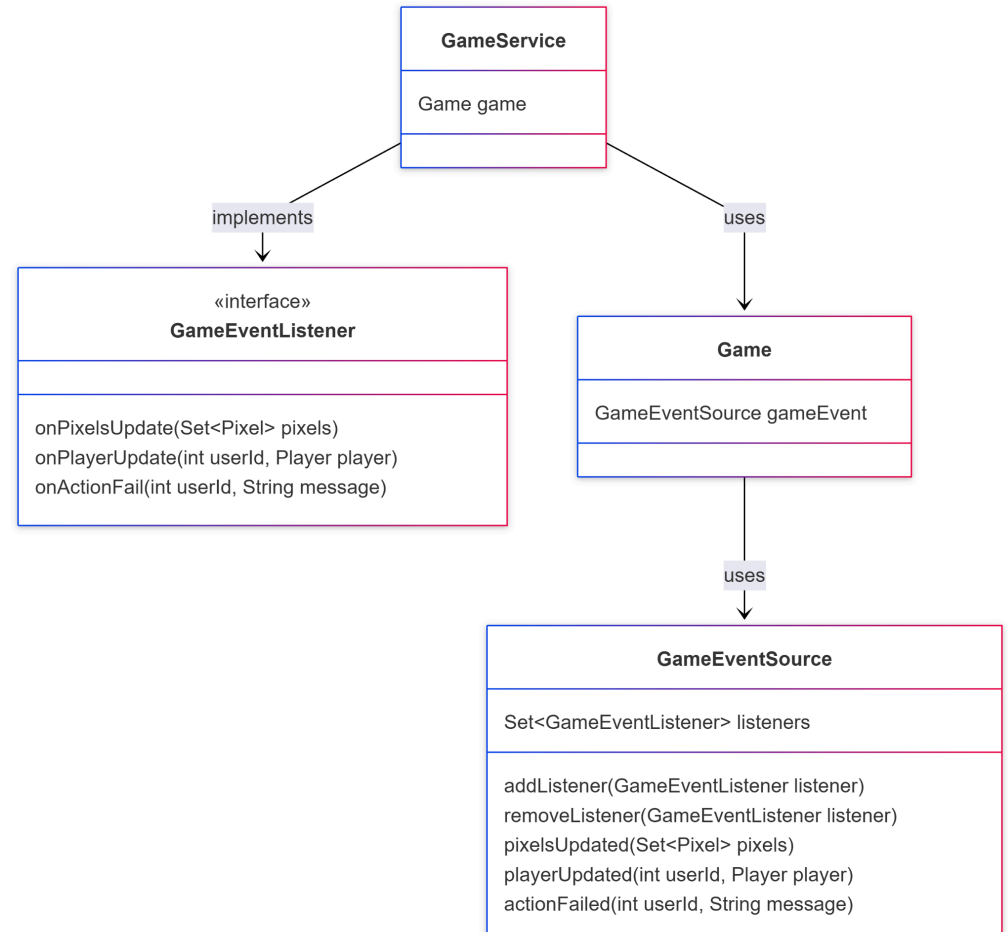


Diagramme UML - Les événements

Websocket

La classe **PlayerSocketJavalin**, dans le dossier /websockets sert de communication entre le backend et le frontend.

Par nature le Websocket est bi directionnel, c'est pourquoi nous l'avons traité comme un adaptateur entrant et sortant.

D'un côté il fait appel aux fonctions métier de **GameService** et de l'autre il implémente l'interface **PlayerNotifieur**.

On note que **GameService** utilise **PlayerNotifieur**, c'est attendu car son rôle est d'orchestrer la logique de jeu, dont informer les joueurs que l'état du jeu à changé.

Cela pose un léger problème d'initialisation, on ne peut pas injecter toutes les dépendances dans les constructeurs car au final **GameService** à besoin de **PlayerSocketJavalin** et inversement.

C'est pourquoi nous avons la méthode `.setPlayerNotifieur()` dans **GameService**.

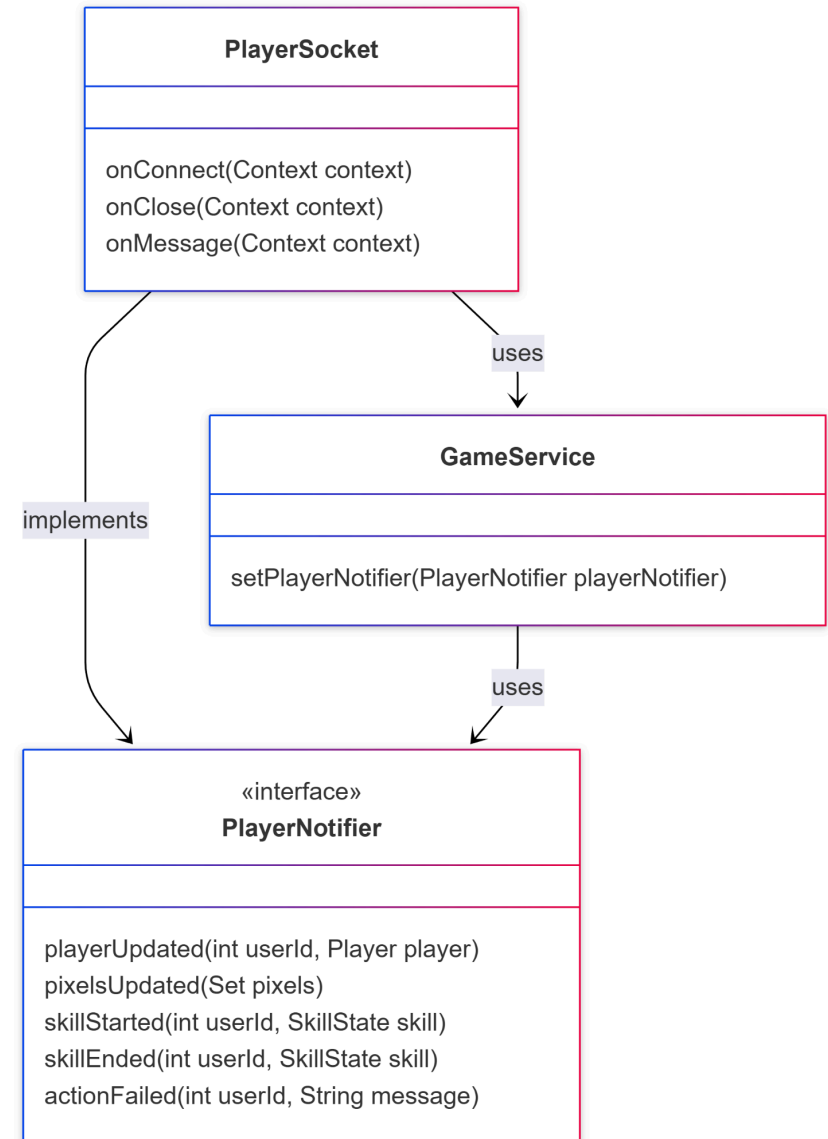


Diagramme UML - PlayerSocket

Frontend